

Shared Memory Cryptographic Vault

<https://github.com/LeoDumas>

Cross-language (C++/Rust) secure vault. Implements custom IPC protocols, OpenSSL envelope encryption, and low-latency shared memory synchronization.

Contents

1. shm-crypt-vault	3
1.1. Project Overview	3
1.2. Architecture	3
1.2.1. Component Separation	3
1.2.2. Inter-Process Communication	3
1.3. Security Model	4
1.3.1. Key Isolation	4
1.3.2. Process Privilege Separation	4
1.3.3. Attack Surface Reduction	4
1.4. Technical Implementation	4
1.4.1. Worker Implementation (C++)	4
1.4.2. Guardian Implementation (Rust)	5
1.4.3. Shared Memory Management	5
1.4.4. Error Handling and Resilience	5
1.5. Use Cases	6
1.5.1. Secure File Archival	6
1.5.2. Data Processing Pipelines	6
1.5.3. Multi-User Environments	6
1.6. Performance Considerations	6
1.7. Deployment and Configuration	6
1.8. Limitations and Considerations	7

1. shm-crypt-vault

1.1. Project Overview

shm-crypt-vault is a security-focused file encryption system that implements a strict separation of concerns between file processing operations and cryptographic key management. The architecture consists of two isolated processes: a Worker component written in C++ that handles bulk file operations, and a Guardian component written in Rust that maintains exclusive control over cryptographic keys and sensitive operations.

The fundamental design principle is that the Worker never possesses encryption keys directly. Instead, it delegates all key-dependent operations to the Guardian through inter-process communication (IPC). This separation minimizes the attack surface by ensuring that even if the Worker process is compromised, cryptographic material remains inaccessible.

1.2. Architecture

1.2.1. Component Separation

The system enforces a clear boundary between computational workload and security-critical operations:

Worker (C++): Responsible for file I/O operations, data buffering, and coordinating the encryption/decryption pipeline. It processes large files in chunks, manages memory efficiently, and handles the user-facing aspects of file operations. The Worker has no access to raw key material and cannot perform cryptographic operations independently.

Guardian (Rust): Acts as a security barrier that exclusively manages cryptographic keys, derives session keys, and performs encryption/decryption of data blocks. It operates as a separate process with restricted privileges and minimal external dependencies. The Guardian validates all requests from the Worker and maintains audit logs of sensitive operations.

1.2.2. Inter-Process Communication

Communication between Worker and Guardian occurs through a defined IPC mechanism. The implementation can leverage shared memory for high-throughput data transfer while using a separate control channel for command and response messages.

Control Channel: Messages for operation requests (encrypt/decrypt), key derivation requests, and status responses. This channel uses a synchronous request-response pattern to ensure ordered operations.

Data Channel: Shared memory segments for passing file chunks between processes. The Worker writes plaintext or ciphertext blocks to shared memory regions, and the Guardian reads from or writes to these same regions without data copying overhead.

The IPC protocol defines message structures for operation types, buffer identifiers, and error codes. Each request includes a nonce or sequence number to prevent replay attacks and ensure message ordering.

1.3. Security Model

1.3.1. Key Isolation

The Guardian maintains cryptographic keys in memory that is never accessible to the Worker process. Keys are derived from user credentials using key derivation functions like PBKDF2 or Argon2, and these derived keys remain exclusively within the Guardian's memory space.

For encryption operations, the Worker requests the Guardian to process a data block by providing a shared memory identifier. The Guardian reads the plaintext from shared memory, encrypts it using the appropriate key, and writes the ciphertext back to the same or different shared memory region. The Worker then writes this ciphertext to the output file without ever seeing the encryption key.

Decryption follows the reverse process: the Worker loads ciphertext into shared memory, requests decryption from the Guardian, and retrieves the plaintext result without accessing keys directly.

1.3.2. Process Privilege Separation

The Guardian operates with minimal privileges necessary only for cryptographic operations. It does not require file system access beyond initial configuration loading. The Worker runs with standard user privileges for file operations but cannot escalate privileges or access Guardian memory.

Operating system process isolation mechanisms (memory protection, separate address spaces) enforce this separation at the kernel level. Even if the Worker process is compromised through a buffer overflow or code injection vulnerability, the attacker cannot directly extract keys from the Guardian process.

1.3.3. Attack Surface Reduction

By limiting the Worker's capabilities, the system reduces potential attack vectors. The Worker handles untrusted input (file paths, user data) but cannot misuse cryptographic material even if exploited. The Guardian exposes only a minimal API surface through IPC, and each operation is validated against expected patterns.

1.4. Technical Implementation

1.4.1. Worker Implementation (C++)

The Worker implements file processing logic using modern C++ features for memory safety and performance. It manages file streams, buffers data in fixed-size chunks (e.g., 64KB or 1MB blocks), and coordinates with the Guardian through the IPC interface.

Key responsibilities include:

- Opening and validating input/output file paths
- Reading files in chunks to manage memory usage for large files
- Allocating and managing shared memory regions for data transfer
- Sending operation requests to the Guardian with appropriate buffer references
- Writing processed data to output files
- Handling errors and cleanup operations

The Worker uses RAII principles to ensure proper resource management and implements exception-safe code to handle failures gracefully.

1.4.2. Guardian Implementation (Rust)

The Guardian leverages Rust's memory safety guarantees and the `ring` or `RustCrypto` libraries for cryptographic operations. It operates as a daemon process that listens for IPC requests and processes them sequentially or with controlled concurrency.

Core functionality includes:

- Loading and deriving cryptographic keys from user credentials
- Managing key lifecycle and secure key erasure on termination
- Processing encryption/decryption requests by accessing shared memory
- Implementing cryptographic operations (AES-GCM, ChaCha20-Poly1305, etc.)
- Maintaining operation logs for audit purposes
- Validating request parameters and enforcing rate limits

The Guardian uses Rust's ownership system to prevent key material from being accidentally copied or exposed through improper references.

1.4.3. Shared Memory Management

Both processes access shared memory regions created by the Worker or through a shared memory manager. Each region is identified by a unique handle that the Worker includes in IPC requests.

Memory regions are mapped with appropriate permissions: the Worker may have read-write access for loading data and retrieving results, while the Guardian maps regions as needed for the duration of each operation. After processing, the Guardian unmaps the memory to minimize exposure.

Synchronization mechanisms (semaphores or condition variables) ensure that the Worker does not access shared memory while the Guardian is processing, preventing race conditions.

1.4.4. Error Handling and Resilience

The system implements comprehensive error handling at multiple levels. The IPC protocol defines error codes for various failure scenarios: invalid requests, cryptographic errors, memory allocation failures, and permission violations.

The Worker handles Guardian unavailability by attempting reconnection with exponential backoff. If the Guardian terminates unexpectedly, the Worker can detect this and fail safely without leaving sensitive data exposed.

The Guardian validates all inputs before processing and returns specific error codes rather than generic failures. This allows the Worker to provide meaningful feedback to users or applications.

1.5. Use Cases

1.5.1. Secure File Archival

Organizations can use shm-crypt-vault to encrypt sensitive documents for long-term storage. The Worker processes files quickly while the Guardian ensures that encryption keys remain isolated from the file processing pipeline.

1.5.2. Data Processing Pipelines

Applications that process confidential data can integrate shm-crypt-vault to add encryption at specific pipeline stages. The Worker can be invoked as a subprocess or library, while the Guardian runs as a system service with elevated security controls.

1.5.3. Multi-User Environments

In shared computing environments, the Guardian can implement key management policies specific to each user. The Worker operates under user context, but the Guardian enforces access controls and key derivation based on authenticated user credentials.

1.6. Performance Considerations

The shared memory approach minimizes data copying between processes, which is critical for processing large files efficiently. For a 1GB file processed in 1MB chunks, the system performs 1024 IPC round-trips plus shared memory operations rather than copying gigabytes of data through pipes or sockets.

The Worker can process multiple files concurrently by managing separate shared memory regions and sending parallel requests to the Guardian. The Guardian can implement a thread pool to handle concurrent requests, though care must be taken to protect shared key material with appropriate locking mechanisms.

Latency for each encryption/decryption operation includes IPC overhead (typically microseconds for local communication), shared memory access, and cryptographic operation time. Modern hardware AES acceleration (AES-NI) can achieve multi-gigabyte per second throughput, making cryptographic operations faster than disk I/O for many use cases.

1.7. Deployment and Configuration

The Guardian process starts independently, typically as a system service or user daemon. It loads configuration specifying allowed operations, logging policies, and security parameters. The Worker connects to the Guardian through a known IPC endpoint (Unix domain socket, named pipe, or shared memory identifier).

Configuration files define cryptographic parameters (algorithms, key sizes, iteration counts for key derivation) and operational settings (maximum file sizes, concurrent operations, timeout values). Both components validate configuration on startup and fail securely if misconfigured.

1.8. Limitations and Considerations

The system's security depends on proper process isolation provided by the operating system. Privilege escalation vulnerabilities in the OS or hypervisor could potentially bypass the separation model.

Shared memory itself does not provide encryption, so data exists in plaintext within shared regions during processing. An attacker with root access or physical memory access could potentially extract this data. The architecture mitigates this by minimizing the time data remains in shared memory and ensuring keys are never present in shared regions.

The IPC mechanism introduces latency compared to in-process encryption. For very small files or operations where the overhead exceeds the encryption time, this architecture may be less efficient than monolithic implementations.